

TECHNICAL ARCHITECTURE REPORT

SYNCLIME

A Cross-Platform Native Desktop Media Extraction Engine
Built on Tauri v2, SolidJS, Rust 2021 Edition, and SQLite

Author Ahmed Trooper
Version 1.0.0
Date July 2026
Repository github.com/AhmedTrooper/Synclime
License MIT / Apache 2.0 (Dual)
Platforms Windows x64/ARM64 • macOS ARM64 • Linux x64/ARM64

TECHNOLOGY STACK

Frontend: SolidJS 1.9, TailwindCSS 4, @solidjs/router, Kobalte UI

Backend: Rust 2021, Tokio, rusqlite (bundled), Axum, parking_lot

Bridge: Tauri v2 IPC (invoke/emit), Tauri Plugin ecosystem

Toolchain: yt-dlp, FFmpeg, aria2c, Bun, Vite, GitHub Actions

This document describes the complete systems architecture, engineering decisions, data-flow design, and concurrency models of the Syncline desktop application.

Produced from a complete file-by-file reading of every source file in the repository.

All architectural details reflect the actual implementation.

Contents

1	Complete System Architecture Diagram	2
2	Executive Overview	4
2.1	Key Capabilities	4
3	Rust Backend Design	5
3.1	Central State Object: AppState	5
3.2	Concurrency Model: The Semaphore Pool	5
3.3	The 1-Second Progress Flush Architecture	6
3.4	Safe Process Cancellation via mpsc Channel	6
3.5	Cookie Security: RAII Drop Guard	7
3.6	Resilient Progress Parsing: Hand-Written Character Scanners	7
4	Database Architecture	7
4.1	Schema Design	8
4.2	Foreign Key Integrity	8
4.3	Performance Indexes	8
4.4	The app_fallback Sentinel Pattern	8
5	IPC Command Architecture	9
5.1	Command Module Map	9
5.2	Job Configuration Resolution: Multi-Level SQL JOIN	9
6	Frontend Architecture	9
6.1	SolidJS Reactive Stores	9
6.2	Real-Time Event Architecture	9
6.3	Custom TitleBar and Window Safety	10
7	Embedded Axum API Server	10
8	CI/CD Pipeline and Cross-Platform Build Matrix	10
9	Security Model	11
9.1	Tauri Capability Model (Least Privilege)	11
9.2	Cookie File Security	11
10	Engineering Decisions Summary	11
11	Codebase Metrics	12
12	Conclusion	12

1. Complete System Architecture Diagram

The following diagram presents the full component hierarchy of Synclime, from the smallest individual modules up through layers of increasing abstraction to the top-level runtime boundary. Each colour identifies a distinct architectural tier. Arrows denote data flow direction; double-headed arrows indicate bidirectional communication.

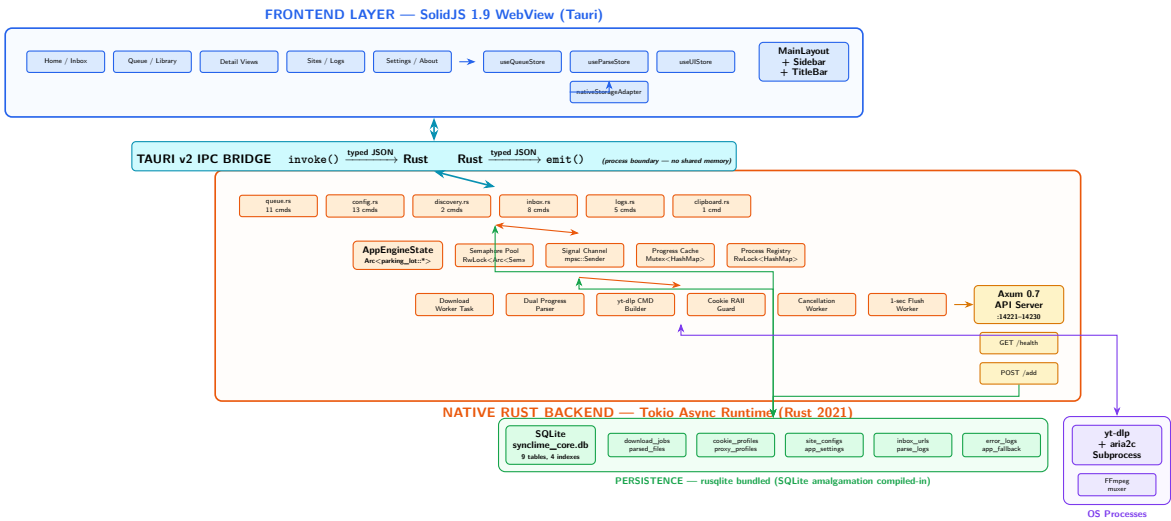


Figure 1: **Complete Syncline Component Architecture.** Colour key: ■ Frontend (SolidJS), ■ IPC Bridge (Tauri), ■ Rust Backend (Tokio), ■ Persistence (SQLite), ■ OS Subprocesses, ■ Embedded API (Axum).

2. Executive Overview

Syncline is a production-grade cross-platform native desktop application engineered to solve a concrete, non-trivial systems problem: orchestrating high-volume media downloads from the internet without freezing the user interface, corrupting shared state, or leaking OS-level processes.

The application ingests video, audio, subtitle, and direct document URLs; resolves metadata via an integrated `yt-dlp` subprocess pipeline; persists download job configurations in a relational SQLite database with enforced foreign-key integrity; and orchestrates concurrent download workers through a Tokio-backed async runtime with a bounded semaphore pool. Real-time progress telemetry is debounced through an in-memory progress cache, flushed to both the database and the UI at 1-second intervals—a deliberate design trade-off that eliminates the most common cause of UI thread freezes in download-manager applications.

Central Design Constraint

The entire architecture flows from one core constraint: *the UI must never block on I/O, the database must never absorb unbounded write storms, and OS-level processes must be safely killable at any moment.* Every structural decision documented below is a direct consequence of that constraint.

The application ships on five binary targets (Windows x64/ARM64, macOS ARM64, Linux x64/ARM64) assembled automatically through a GitHub Actions matrix pipeline with platform-specific native dependency management.

2.1. Key Capabilities

- **Concurrent download queue:** Semaphore-bounded worker pool, dynamically reconfigurable at runtime without restarting any running jobs.
- **Debounced progress telemetry:** In-memory HashMap absorbs thousands of `yt-dlp` stdout lines per second; a background Tokio task flushes to SQLite and emits one Tauri event per job per second.
- **Safe process cancellation:** Pause signals route through an mpsc channel to a dedicated cancellation worker; on Unix, whole process groups are killed (`kill -9 -<pgid>`) to eliminate aria2c zombie children.
- **Cookie-secured downloads:** Netscape-format session cookies are written to 0o600-permission temporary files and deleted by a Rust `Drop` guard after each job, regardless of outcome.
- **Browser extension inbox:** An embedded Axum HTTP server listens on `127.0.0.1:14221--14230` and persists URLs from companion extensions directly to SQLite with real-time badge updates on the UI.
- **Domain routing rules:** Per-domain site configurations associate cookie vaults and SOCKS5/HTTP proxy profiles via SQLite foreign keys, with live validation badges detecting broken credential references.
- **Dual-format progress parsing:** ANSI-stripped `yt-dlp` lines and aria2c parenthetical percentages are decoded by hand-written character-level scanners—not regex—for robustness against format

variations.

- **Full audit logging:** Parse operations and download errors are persisted to `parse_logs` and `error_logs` with executed commands, exit codes, byte counts, and RFC 3339 timestamps.

3. Rust Backend Design

3.1. Central State Object: `AppEngineState`

All shared mutable application state is consolidated into one `AppEngineState` struct managed by Tauri’s `app.manage()` API. Every command handler receives it as an injected `State<'_, AppEngineState>`—zero-copy access without constructor injection boilerplate.

```
1 pub struct AppEngineState {
2     pub pool_semaphore: Arc<parking_lot::RwLock<Arc<Semaphore>>>,
3     pub db_path: std::path::PathBuf,
4     pub db_conn: Arc<parking_lot::Mutex<Connection>>,
5     pub active_processes: Arc<ActiveProcessRegistry>,
6     pub signal_tx: mpsc::Sender<QueueSignal>,
7     pub progress_cache: Arc<parking_lot::Mutex<HashMap<String, ProgressSnapshot>>>,
8 }
```

Listing 1: `AppEngineState` (lib.rs)

Why `parking_lot` over `std::sync`?

`parking_lot::Mutex` and `RwLock` are smaller, faster under contention, and lack the poisoning semantics of the standard library (panics in a holding thread do not corrupt shared state for other threads). For a data path hit once per second by the flush worker and on-demand by every command handler, this is a deliberate performance choice.

The `pool_semaphore` is wrapped as `RwLock<Arc<Semaphore>>` rather than a plain `Semaphore`. This enables the concurrency limit to be hot-swapped at runtime: the `update_concurrency_limit` command writes a new `Arc<Semaphore>` under the write lock without invalidating any currently-held permits in running workers.

3.2. Concurrency Model: The Semaphore Pool

Download workers are dispatched as detached Tokio tasks:

```
1 tauri::async_runtime::spawn(async move {
2     let _ = execute_download_worker(app_handle, worker_slug).await;
3 });
```

Listing 2: Non-blocking task dispatch (commands/queue.rs)

Inside `execute_download_worker`, the worker acquires a semaphore permit before spawning the OS process. The permit is held for the full lifetime of the download—released when the function returns. No more than N concurrent `yt-dlp` processes can exist, where N is the user-configured concurrency limit. The owned permit pattern guarantees release through RAII even on panics or early returns.

3.3. The 1-Second Progress Flush Architecture

A naive implementation would write every yt-dlp stdout line directly to SQLite and emit a Tauri event per line—thousands of writes per second per download, which reliably causes UI freezes and database lock contention.

Syncline resolves this with a two-stage pipeline:

1. **Stage 1 (per stdout line):** Progress data is written to a `HashMap<String, ProgressSnapshot>` under a `Mutex`—a sub-microsecond in-memory operation.
2. **Stage 2 (once per second):** A background Tokio task wakes, locks the cache, bulk-writes all snapshots to SQLite, emits one Tauri event per active job, then clears the cache.

```

1 tauri::async_runtime::spawn(async move {
2     loop {
3         tokio::time::sleep(Duration::from_secs(1)).await;
4         let mut cache = flush_cache.lock();
5         if !cache.is_empty() {
6             let conn = flush_conn.lock();
7             for (slug, snap) in cache.iter() {
8                 let _ = conn.execute(
9                     "UPDATE download_jobs
10                      SET progress=?1, tracking_message=?2, status=?3,
11                        updated_at=datetime('now')
12                      WHERE slug=?4;",
13                     params![snap.progress, snap.status_message, snap.status, slug],
14                 );
15                 let _ = tic_emitter.emit("download-progress-token",
16                     json!({
17                         "slug": slug, "progress": snap.progress,
18                         "message": &snap.status_message, "status": &snap.status
19                     }));
20             }
21             cache.clear();
22         }
23     }
24 }

```

Listing 3: 1-second flush worker (lib.rs)

3.4. Safe Process Cancellation via mpsc Channel

```

1 async fn start_cancellation_worker(
2     mut rx: mpsc::Receiver<QueueSignal>,
3     registry: Arc<ActiveProcessRegistry>,
4 ) {
5     while let Some(QueueSignal::PauseJob(slug)) = rx.recv().await {
6         let process = {
7             let mut instances = registry.instances.write();
8             instances.remove(&slug)
9         };
10        if let Some(mut child) = process {
11            #[cfg(unix)]
12            if let Some(pid) = child.id() {
13                // Kill entire process GROUP: yt-dlp + any aria2c children
14                let _ = Command::new("kill")
15                    .args(&["-9", &format!("{}", pid)])
16                    .status();
17            }
18            #[cfg(not(unix))]

```

```

19         let _ = child.kill().await;
20     }
21 }
22 }

```

Listing 4: Cancellation worker (lib.rs)

On Unix, `kill -9 -<pgid>` terminates the entire process group, preventing aria2c fragment workers from becoming zombie processes after yt-dlp itself exits.

3.5. Cookie Security: RAII Drop Guard

```

1 struct CookieFileCleanup(Option<std::path::PathBuf>);
2 impl Drop for CookieFileCleanup {
3     fn drop(&mut self) {
4         if let Some(ref path) = self.0 {
5             let _ = std::fs::remove_file(path); // runs on success, error, or panic
6         }
7     }
8 }
9 let _cleanup_guard = CookieFileCleanup(temp_cookie_path.clone());

```

Listing 5: Cookie file RAII cleanup (engine/process.rs)

3.6. Resilient Progress Parsing: Hand-Written Character Scanners

Rather than brittle regex, Syncline uses two hand-written character-level scanners:

`parse_progress_line`

Strips ANSI sequences character-by-character using an `in_ansi` state machine flag, then backward-scans from the first `%` to extract the numeric percentage.

`parse_aria2_progress`

Scans all `%` characters, backward-searches for the nearest `(`, and extracts aria2c's (99%) format.

```

1 let parsed_progress = parse_progress_line(clean_line)
2   .or_else(|| parse_aria2_progress(clean_line));

```

Listing 6: Dual-parser chaining (engine/process.rs)

Four unit tests in `#[cfg(test)] mod tests` cover standard output, whitespace edge cases, ANSI-prefixed lines, and non-progress lines.

4. Database Architecture

4.1. Schema Design

Table	Purpose	Key Relations
cookie_profiles	Netscape-format cookie blobs per domain	—
proxy_profiles	SOCKS5/HTTP proxy endpoint strings	—
site_configs	Per-domain routing rules	FK → cookie, proxy
app_settings	KV store: paths, limits, active port	—
parsed_files	yt-dlp extracted metadata manifests	FK → site_configs, self
download_jobs	Queue items with status/progress	FK → parsed_files, cookies, proxy, self
parse_logs	Audit log for extraction runs	FK → parsed_files (CASCADE)
error_logs	Per-job error messages + commands	FK → download_jobs (CASCADE)
inbox_urls	Browser extension ingestion buffer	—

Table 1: Database schema summary (9 tables)

4.2. Foreign Key Integrity

`PRAGMA foreign_keys = ON` is executed on every connection open. Referential actions are tuned per relationship:

- `site_configs` → `credentials`: `ON DELETE SET NULL`
- `download_jobs` → `download_jobs` (self): `ON DELETE CASCADE`
- `parse_logs` / `error_logs`: `ON DELETE CASCADE`

4.3. Performance Indexes

```

1 CREATE INDEX IF NOT EXISTS idx_parsed_files_parent
2   ON parsed_files (parent_playlist_slug)
3   WHERE parent_playlist_slug IS NOT NULL;
4
5 CREATE INDEX IF NOT EXISTS idx_download_jobs_parsed_file
6   ON download_jobs (parsed_file_slug)
7   WHERE parsed_file_slug IS NOT NULL;
8
9 CREATE INDEX IF NOT EXISTS idx_download_jobs_subtitles
10  ON download_jobs (associated_media_job_slug)
11  WHERE file_type = 'subtitle' AND associated_media_job_slug IS NOT NULL;
12
13 CREATE INDEX IF NOT EXISTS idx_download_jobs_queue_priority
14  ON download_jobs (status, priority_index DESC, created_at ASC);

```

Listing 7: Partial indexes (lib.rs)

4.4. The `app_fallback` Sentinel Pattern

On startup, sentinel rows with slug `'app_fallback'` are inserted into both `parsed_files` and `download_jobs` via `INSERT OR IGNORE`. Error logging commands fall back to this sentinel if the target job has been deleted—preventing foreign key violations from silently swallowing diagnostic data.

5. IPC Command Architecture

5.1. Command Module Map

Module	Cmds	Responsibility
commands/queue.rs	11	Job lifecycle: start, pause, insert, delete, concurrency settings
commands/config.rs	13	Cookie/proxy profiles, site configs, download path
commands/discovery.rs	2	yt-dlp metadata extraction, parsed file persistence
commands/inbox.rs	8	Inbox CRUD, remote update fetch, API port query
commands/logs.rs	5	Error/parse log retrieval, insertion, bulk clear
commands/clipboard.rs	1	URL validation + UTM tracking parameter stripping

Table 2: Tauri IPC command module map

5.2. Job Configuration Resolution: Multi-Level SQL JOIN

```

1 SELECT
2     COALESCE(j.direct_url, p.url) as target_url,
3     COALESCE(sc_cookie.cookie_data, c.cookie_data) as cookie_data,
4     COALESCE(sc_proxy.proxy_string, pr.proxy_string) as proxy_string, ...
5 FROM download_jobs j
6 LEFT JOIN parsed_files p ON j.parsed_file_slug = p.slug
7 LEFT JOIN parsed_files parent_p ON p.parent_playlist_slug = parent_p.slug
8 LEFT JOIN site_configs sc ON
9     COALESCE(p.site_config_slug, parent_p.site_config_slug) = sc.slug
10 LEFT JOIN cookie_profiles sc_cookie ON sc.cookie_profile_slug = sc_cookie.slug
11 LEFT JOIN proxy_profiles sc_proxy ON sc.proxy_profile_slug = sc_proxy.slug
12 LEFT JOIN cookie_profiles c ON j.cookie_profile_slug = c.slug
13 LEFT JOIN proxy_profiles pr ON j.proxy_profile_slug = pr.slug
14 WHERE j.slug = ?1;

```

Listing 8: Multi-join config resolution (engine/process.rs)

Priority: **1**) Direct per-job override; **2**) Site config on the parsed file; **3**) Site config inherited from parent playlist. Resolved in a single round-trip.

6. Frontend Architecture

6.1. SolidJS Reactive Stores

Store	State Shape	Persistence
useQueueStore	DownloadJob[], progress map	In-memory; refreshed from SQLite
useParseStore	ParsedFile[], isParsing	Tauri Store plugin (binary file)
useUIStore	theme, path, badges, sidebar	Tauri Store plugin (binary file)

Table 3: Frontend reactive stores

6.2. Real-Time Event Architecture

```

1 import("@tauri-apps/api/event").then(({ listen }) => {
2   listen("download-progress-token", (event: any) => {
3     const { slug, progress, message, status } = event.payload;
4     useQueueStore.updateJobProgress(slug, progress, message);
5     if (status) useQueueStore.updateJobStatus(slug, status as any);
6   });
7 });

```

Listing 9: Global event listener (store/useQueueStore.ts)

6.3. Custom TitleBar and Window Safety

The Tauri window uses `"decorations": false`, `"transparent": true`. The *Quit App* button triggers a native OS `ask()` dialog warning active downloads will be interrupted. F5, Ctrl+R, and DevTools shortcuts are intercepted and blocked at the `keydown` level to prevent accidental WebView state destruction.

7. Embedded Axum API Server

Endpoint	Method	Input	Effect
/health	GET	—	JSON status, version
/add	POST	{ <code>"url": "..."</code> }	INSERT to <code>inbox_urls</code> ; emit <code>inbox-updated</code>

Table 4: Axum local API endpoints

Port range 14221--14230 is scanned for the first available binding; the active port is persisted to `app_settings` for frontend discovery. The `/add` handler shares the same `Arc<Mutex<Connection>>` as the rest of the application, preserving the single-writer SQLite model.

8. CI/CD Pipeline and Cross-Platform Build Matrix

Runner	Rust Target	Notes
macos-latest	aarch64-apple-darwin	Apple Silicon
ubuntu-22.04	x86_64-unknown-linux-gnu	x64 Linux
ubuntu-22.04-arm	aarch64-unknown-linux-gnu	ARM64 Linux
windows-latest	x86_64-pc-windows-msvc	Windows x64
windows-latest	aarch64-pc-windows-msvc	Windows ARM64

Table 5: GitHub Actions 5-target build matrix

- **Linux:** apt installs WebKit2GTK, HarfBuzz, Graphite2, ICU; `RUSTFLAGS` force-links `-lstdc++` and `-lgraphite2`.
- **macOS:** Homebrew installs icu4c and libpng; `PKG_CONFIG_PATH` covers both Intel and Apple Silicon prefix locations.

- **Windows:** vcpkg installs ICU, HarfBuzz, Graphite2, fontconfig with architecture-aware triplets; CXXFLAGS=/std:c++17 set for MSVC.

9. Security Model

9.1. Tauri Capability Model (Least Privilege)

`capabilities/default.json` explicitly enumerates the permissions granted to the main window. No filesystem read/write capability is granted to the WebView; all file access is mediated through typed Rust commands.

9.2. Cookie File Security

- Unix: 0o600 (owner-read-write only) via `OpenOptionsExt`
- RAII Drop guard deletes the file after the download exits—success, error, or panic
- Startup sweep removes any `synclime_cookie_*.txt` leftover from crashes
- Nanosecond-timestamped filenames prevent concurrent-download collisions

10. Engineering Decisions Summary

Key Technical Choices and Their Rationale

SolidJS over React

Fine-grained reactivity: only the changed signal re-renders. For a queue with sub-second progress updates, this is measurably better than Virtual DOM diffing.

rusqlite bundled feature

SQLite amalgamation compiled into the binary: zero runtime dependency on system SQLite, identical behaviour across all five target platforms.

parking_lot over std::sync

Lower lock overhead under contention; no mutex poisoning semantics.

mpsc channel for cancellation

Decouples the IPC command handler (must return fast) from blocking OS kill calls.

Arc<RwLock<Arc<Semaphore>>>

Enables atomic hot-swap of the semaphore pool when the concurrency limit changes, without disrupting currently-acquiring workers.

Character-level progress scanners

Immune to ANSI escape sequence variants and Unicode edge cases that would break regex.

Bun + frozen lockfile

Deterministic CI builds; Bun's native TypeScript execution speeds the development loop.

Port range scanning 14221–14230

Handles port conflicts gracefully without user configuration.

11. Codebase Metrics

Layer	Files	Approx. Lines	Language
Rust backend (lib.rs + 5 cmd modules + engine)	11	~2,600	Rust
SolidJS routes (12 page components)	12	~3,600	TypeScript/TSX
Layouts & shared components	4	~700	TypeScript/TSX
Reactive stores & storage adapter	4	~280	TypeScript
Core types & logger	4	~280	TypeScript
Configuration & CI	6	~350	—
Total	41	~7,800	

Table 6: Codebase metrics by architectural layer

12. Conclusion

Syncline demonstrates a coherent set of senior engineering practices applied consistently across a full-stack native desktop application:

- **Concurrent systems design:** Semaphore-bounded pools, mpsc cancellation channels, and debounced flush architecture prevent UI freeze, ghost processes, and database write storms.
- **Relational data modelling:** Normalised schema with foreign key integrity, partial indexes, carefully chosen referential actions, and sentinel pattern for orphan-safe audit logging.
- **Security by design:** Least-privilege Tauri capabilities, RAII-guarded temporary secrets, process group termination on Unix, startup artifact cleanup.
- **Resilient parsing:** Hand-written character scanners over fragile regex for progress output subject to ANSI codes and format variations.
- **Cross-platform delivery:** 5-target CI/CD matrix with platform-specific native dependency management producing native installers automatically.
- **Integration depth:** Embedded Axum server, reactive event pipeline, and dual-backend storage adapter create a seamless flow from browser capture to download completion with no polling at any layer.

Architecture Quality Signal

The codebase is free of several patterns diagnostic of intermediate-level work: no global mutable statics, no `unwrap()` in production paths (errors propagate via `Result<T, String>`), no raw thread spawning (all concurrency through Tokio), no shared state without synchronisation primitives, and no sleep-loop polling for coordination. The architecture is *reactive at both ends*: Rust pushes typed events; SolidJS consumes them through a module-level listener registered once. Neither side polls.

The result is a production-grade native desktop application that handles real concurrency and real failure modes with principled, deliberate engineering.

Syncline Technical Architecture Report — v1.0.0 Author: Ahmed Trooper Repository: github.com/AhmedTrooper/Syncline
License: MIT / Apache 2.0 (Dual) Generated: July 2026